stricter about physical layout. For example, Python is known for enforcing the use of indentation to denote logical blocks of code. Ruby, for better or worse, is not like that. Ruby offers the developer tremendous flexibility.

However, in the face of that flexibility, various common practices have emerged. In some cases, a particular style was found to have genuine clarity or modification benefits. In other cases, the community picked from equally fine choices, but the consistency of going with only one choice has its own benefit.

Currently, the two most commonly used Ruby style guides can be found at https://rubystyle.guide, which is maintained by Bozhidar Batsov and the team that maintains the RuboCop tool, and at https://ruby-style-guide.shopify.dev, which is a slightly different version maintained by the team of Ruby developers at Shopify.

It is not our intention to litigate every element in those guides, but we do want to mention a few of the most important or most widely observed features. (You'll notice the features we highlight have a strong correlation with the ones used by the Standard tool.)

Indentation and Spacing

Ruby code almost without exception uses two spaces for indentation—not tabs and not four spaces:

```
module Game
  class Team
    def initialize
    end
  end
end
```

We could go on about how this is optimal, but honestly, it's largely a question of consistency (and to some extent, a preference for more compact code).

It's important to note that a couple of things are not indented in Ruby that might be indented in other languages. For example, in a case statement, the `when` clauses are not indented relative to the `case`:

```
case name
when :superman
  "Clark Kent"
when :batman
  "Bruce Wayne"
end
```

Similarly, exception keywords like `rescue` and `ensure` are not indented relative to the blocks they are inside:

```
def a_sample_method(name)
  a_dangerous_call
rescue StandardError
  return "Oops"
ensure
  something_else
end
```